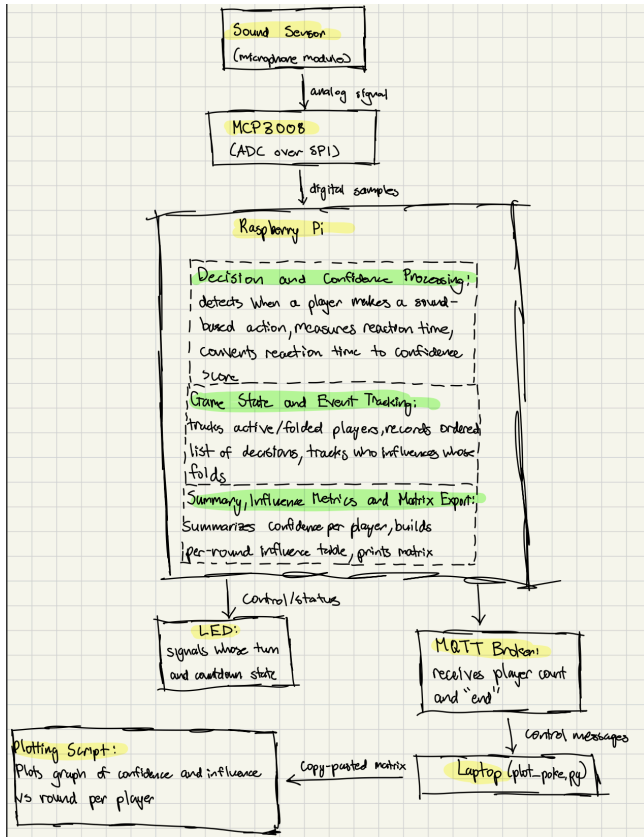


Poke the Poker Documentation

Description

Poke the Poker is a real-time poker analyzer designed to track players' decisions, confidence, and influence on one another. The system relies on a sound sensor and converts raw analog data into digital outputs to compute confidence and influence levels. By the end of the game, Poke the Poker shows a comprehensive table and graph(s) showing player(s) confidence and influence scores over the course of the game. This can best be used by commentators, moderators, or analysts.

Block Diagram



Components, Protocols, Processing Techniques, Implementation, & Design Choices

MCP3008: Provides the analog to digital conversion required for the GrovePi sound sensor. It connects to the RPi over SPI and gives the system reliable sampling of sound levels. This conversion makes it possible to detect decision events with enough precision to measure timing and compute confidence scores.

GrovePi Sound Sensor: The primary input device. It outputs an analog signal, read through the MCP3008. The system samples the sensor at short intervals and looks for several consecutive readings above a preset threshold. This method reduces false triggers from noise while allowing fast and responsive decision detection.

Raspberry Pi: The Raspberry Pi runs all game logic. It timestamps decisions, converts timing into confidence scores, manages fold states, and logs every decision event with player ID and round information. The RPi also drives an onboard LED that lights up during cooldown periods to tell players to wait before the next turn begins.

MQTT Communication: Provides lightweight communication between the RPi and an external controller. The RPi subscribes to a control topic that provides the initial player count and the optional "end" command. This approach removes the need for a custom user interface and allows game control from any device that can publish MQTT messages, typically a laptop though.

Real Time Processing: The system uses threshold based sound detection to identify decisions and measures the time from the start of the decision window until detection. This duration becomes the confidence score. A timeout produces a confidence of zero and counts as a fold.

Data Logging and Analysis: Each decision is recorded with confidence, fold status, round number, and player ID. After the game ends, the system computes aggressor influence, target based influence coverage, and other metrics. It can also export a matrix of consecutive decisions for the plotting script to visualize confidence and influence trends.

Design Choices: All core processing remains on the RPi, which keeps the game loop stable even if the controller disconnects. MQTT is used only for simple inputs. The game ends automatically when only one active player remains, preventing stalled rounds and keeping gameplay consistent.

Limitations

During testing and Poker games, several limitations became clear during development and testing. One limitation is the sound based input method because the GrovePi sound sensor cannot distinguish between intentional player signals and ambient noise. We reduced false triggers by increasing the sound threshold. However, a more robust approach would involve a microphone with frequency filtering, reducing uncertainty.

Another limitation we encountered involved the connector between the Raspberry Pi and the GrovePi sound sensor because the sound sensor would frequently cut out, causing it to register incomplete or inconsistent sound values, which led to repeated instances where we had to replace the connectors entirely. A potential solution to this issue would be to use higher-quality connectors or to implement a more secure wiring method, such as soldered joints or reinforced jumper cables, to reduce signal loss and improve reliability.

The final limitation is the LED based feedback because it does not give players much information beyond whether they should wait. Instead, we could have used multi color LEDs or small displays, which would help players interpret the state of the game.

Despite these limitations, the project taught us how to integrate sensors, analog-to-digital conversion, MQTT communication, and data analysis into a fully functioning IoT system. We learned how hardware choices can significantly affect signal quality, how timing and sampling rates influence the behavior of real-time systems, and how user interaction becomes a design constraint that shapes the flow of the game. These lessons showed us how we can make improvements in future iterations and make more user-aware design choices.

Open and Transparent Use of AI tools

We used ChatGPT for:

- helping install mosquitto/MQTT communication on the controller side as we used a Windows OS
- looking into data analysis and statistics models we could use for influence scoring
- debugging logic within confidence and influence scoring logic
- output MATRIX for plotting at end of game

[Presentation video](#)